Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis 3
Status- und Bedienkommandos implementieren

event status, stats und reset · native_sim

3.07

Praxisauftrag und Abnahme

Aufbauend auf 3.05: Implementiert werden nur Shell-Handler und ihre Anbindung

01

Modul

event_shell.c anlegen und in CMake einbinden

02

Registrieren

event mit statischen Unterkommandos veröffentlichen

03

Status

Betriebszustand, Target und Bereitschaft ausgeben

04

Statistik und Reset

Zähler anzeigen und mit klarer Semantik zurücksetzen

05

Prüfen

Fehlerfälle, Hilfe und Autovervollständigung testen

Shell-Code verwendet ausschließlich öffentliche Funktionen des Event Service.

Projektstellen und Verantwortungen

EVENT_SHELL.C

CLI-Grenze

Kommandos registrieren

Argumente prüfen

Antworten ausgeben

EVENT_SERVICE.H

Öffentliche API

Status lesen

Statistik lesen

Reset anfordern

TESTMATRIX

Vorbereitet

gültige Aufrufe

fehlende und zusätzliche Argumente

Hilfe und Tab-Vervollständigung

Die Shell bedient den Event Service. Sie enthält keine zweite Implementierung der Fachlogik.

Kommandovertrag in drei Fällen

01	`event status`	Betriebszustand, Target und Gerätebereitschaft ausgeben
02	`event stats`	empfangene, verarbeitete und verworfene Ereignisse anzeigen
03	`event reset`	definierte Zähler atomar auf ihren Startwert setzen

Jedes Unterkommando besitzt eindeutige Syntax, Ausgabe, Fehlerfälle und Seiteneffekte.

Handler-Muster ohne Fachlogik

NUR DIE SCHNITTSTELLE

```
struct event_status status;
int rc = event_service_get_status(&status);
if (rc < 0) { return rc; }

shell_print(sh, "state: %s", status.name);
return 0;
```

01 **Öffentliche API**

Der Handler liest Zustand nur über event_service.h.

02 **Bedienantwort**

shell_print() meldet das Ergebnis des konkreten Aufrufs.

Fehler werden verständlich ausgegeben und als negativer Rückgabewert weitergegeben.

Prüfmatrix für Shell-Kommandos

Jeder Test beginnt mit bekanntem Zustand und prüft Ausgabe, Rückgabewert sowie Seiteneffekt.

01	02	03	04	05
Status	**Statistik**	**Reset**	**Fehler**	**Hilfe**
event status zeigt Zustand	event stats zeigt Zähler	event reset setzt definiert zurück	falsche Argumente ändern keinen Zustand	event -h Tab ergänzt Namen

Die Testmatrix umfasst Erfolg, Nutzungsfehler und einen nicht verfügbaren Service.

CLI-Review und erwartete Ausgaben

`event status`	Zustand und Target sind eindeutig
`event stats`	Zählernamen und Werte sind stabil
`event reset`	Erfolg und Wirkung sind klar
`event reset extra`	Nutzungsfehler, Zustand unverändert
`event -h und Tab`	Syntax und Unterkommandos auffindbar

Shell-Antworten gehören zur Bedienung. Asynchrone Ereignisse bleiben in den Logs.

Abnahme der Shell-Kommandos

FUNKTION

Drei Kommandos verfügbar

status zeigt Zustand

stats zeigt Zähler

reset wirkt definiert

ROBUSTHEIT

Fehler sauber behandelt

fehlend und überzählig geprüft

ungültige Werte abgelehnt

Zustand bleibt konsistent

ARCHITEKTUR

Verantwortung getrennt

Shell validiert und berichtet

Event Service besitzt Zustand

Ausgaben und Logs getrennt

Bestanden ist die Übung, wenn Bedienung, Fehlerwege und Service-Grenze nachvollziehbar sind.